

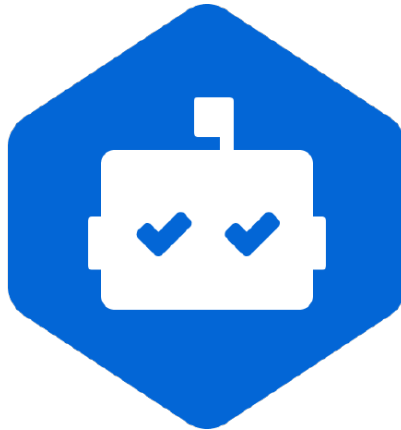


Illustration : logo de Dependabot, © GitHub

Dependabot

Automatiser la mise à jour sécurisée des dépendances sur GitHub

SYNTHÈSE TECHNIQUE



RÉALISÉ PAR (BINÔME)
Madou DIAKAYETEB
Mouhamed [NOM]

SOUS LA DIRECTION DE
[Nom du professeur]

Plan

Cette synthèse part du problème que Dependabot résout, explique ensuite son fonctionnement et sa configuration, puis situe son rôle exact, et ses limites, dans un pipeline CI/CD. Chaque point est illustré par notre dépôt de démonstration, volontairement bloqué sur une version ancienne d'axios.

Introduction	3
1. Dependabot et le problème qu'il résout	3
Ce qu'est Dependabot, ses trois fonctions, et le risque qu'il combat : des dépendances qui deviennent vulnérables sans que le code change (axios 0.18.0, Equifax).	
1.1 Un robot de mise à jour intégré à GitHub	3
1.2 Des dépendances qui vieillissent mal	3
2. Fonctionnement et configuration	3
Comment une faille est repérée, ce qui distingue security updates et version updates, et le fichier dependabot.yml commenté de notre dépôt.	
2.1 Repérer une dépendance vulnérable	3
2.2 Deux types de mises à jour	3
2.3 Le fichier dependabot.yml	4
3. Dependabot dans un pipeline CI/CD	4
Sa place exacte entre détection et décision, la preuve par notre dépôt que le CI est indispensable, et les limites à connaître.	
3.1 Détecter, proposer, vérifier, décider	4
3.2 Limites et points de vigilance	5
Conclusion	5
Sources	5

Fil conducteur

Dependabot **détecte** et **propose** ; le pipeline CI **vérifie** ; l'équipe **décide**.

Introduction

Une application moderne est surtout faite de code écrit par d'autres. Notre application de démonstration (React, Vite, TypeScript) déclare 14 dépendances dans son `package.json`, mais leur installation télécharge **159 paquets** : chaque bibliothèque apporte les siennes (dépendances indirectes, dites « transitives »). Suivre à la main les failles et les nouvelles versions de chacun est irréaliste. **Dependabot**, le robot intégré à GitHub, automatise cette veille : il repère les dépendances vulnérables ou dépassées et propose de les mettre à jour sous forme de *pull requests*. Cette synthèse présente ce qu'il est et le problème qu'il résout (1), son fonctionnement et sa configuration (2), puis son rôle exact et ses limites dans un pipeline CI/CD (3), à partir de notre dépôt de démonstration, volontairement bloqué sur axios 0.18.0.

1. Dependabot et le problème qu'il résout

1.1 Un robot de mise à jour intégré à GitHub

Dependabot lit les fichiers qui déclarent les dépendances d'un projet, le **manifeste** (`package.json`) et le **lockfile** qui fige les versions exactes installées (`package-lock.json`), repère celles qui sont vulnérables ou dépassées, puis ouvre de lui-même une **pull request** (PR) contenant la mise à jour. Service indépendant racheté par GitHub en mai 2019, il est aujourd'hui intégré et gratuit : rien à installer, il s'active dans *Settings* › *Advanced Security* et se règle par un fichier YAML versionné avec le code, `.github/dependabot.yml`. Il couvre une trentaine d'écosystèmes (npm, pip, Maven, Gradle, Go modules, Cargo, NuGet, Docker, GitHub Actions, Terraform...). Sous ce nom se cachent trois fonctions : les **alerts** signalent chaque dépendance touchée par une faille connue ; les **security updates** ouvrent une PR vers la version corrigée ; les **version updates** proposent régulièrement les nouvelles versions, même sans faille.

1.2 Des dépendances qui vieillissent mal

Une faille est souvent découverte des années après la publication d'une version. Publiée le 19 février 2018, axios 0.18.0 a reçu sa première faille en mai 2019 (CVE-2019-10742, déni de service) ; le 25 septembre 2026, **25 avis de sécurité** la concernaient dans la GitHub Advisory Database, dont **11 de gravité élevée**. Le code n'a pas changé ; c'est le risque qui a augmenté. Avec CVE-2025-27152 par exemple, un client axios créé avec une adresse de base (`baseURL`) et une clé d'API secrète envoie la requête, clé comprise, vers un serveur tiers si on lui passe une adresse absolue fournie par un utilisateur.

Gérées à la main, les mises à jour reposent sur la bonne volonté de chacun, et « ça marche, on n'y touche pas » devient la règle. Le cas d'école est **Equifax** : le correctif de la faille Apache Struts CVE-2017-5638, disponible depuis le 7 mars 2017, n'a pas été appliqué sur le portail concerné, exploité de mai à juillet 2017 ; les données d'environ 147 millions de personnes ont été exposées. C'est l'enjeu de la **supply chain security** (sécurité de la chaîne d'approvisionnement logicielle) : s'assurer que les « pièces » d'une application ne sont ni **défectueuses** (failles connues), ni **piégées** (versions malveillantes). Dependabot traite d'abord le premier risque ; son délai d'attente (*cooldown*, section 3.2) réduit le second.

2. Fonctionnement et configuration

2.1 Repérer une dépendance vulnérable

1. **Le graphe de dépendances.** GitHub analyse manifestes et lockfiles et en déduit la liste exacte des dépendances et de leurs versions, transitives comprises.
2. **La GitHub Advisory Database.** Base publique des failles connues (identifiants CVE et GHSA) ; seuls les avis relus par GitHub déclenchent des alertes.
3. **La comparaison.** Si la version installée tombe dans une plage vulnérable (axios « < 0.21.2 » pour CVE-2021-3749), une alerte est créée avec la gravité et la première version corrigée. Elle est refaite à chaque nouvel avis et à chaque push qui modifie les dépendances.

Pour les version updates, le principe est plus simple : à chaque échéance du calendrier, Dependabot compare la version utilisée à la dernière publiée sur le registre (npmjs.org pour npm).

2.2 Deux types de mises à jour

Dependabot ouvre deux sortes de PR, comparées dans le tableau ci-dessous. **Observé dans notre dépôt le 23 septembre 2026** : au premier passage, il a calculé 12 mises à jour de version mais n'a ouvert que 5 PR (limite par défaut), plus 2 PR de sécurité qui échappent à cette limite. Ces traitements tournent sur des runners GitHub Actions (onglet Actions, « Dependabot Updates »). Chaque PR est prête à relire : titre explicite (`build(deps-dev): bump globals from`

15.15.0 to 17.12.0, qui reprend la convention Conventional Commits de notre historique), notes de version et changelog, étiquettes, lien vers l'alerte corrigée, et commandes à écrire en commentaire (@dependabot rebase, @dependabot ignore <dépendance> major version...).

	Security updates	Version updates
Déclencheur	Une alerte (faille connue)	Le calendrier (schedule)
Configuration	Activation dans les paramètres ; dependabot.yml facultatif	dependabot.yml obligatoire
Version proposée	La version minimale qui corrige la faille	La dernière version publiée
Limite de PR	Non soumises à la limite de PR ouvertes	5 PR ouvertes au maximum par défaut
Délai	Immédiat	3 jours après la publication (cooldown par défaut depuis juillet 2026)
Dans notre dépôt	axios 0.18.0 → 0.33.0 (PR n° 6)	axios 0.18.0 → 1.20.0

2.3 Le fichier dependabot.yml

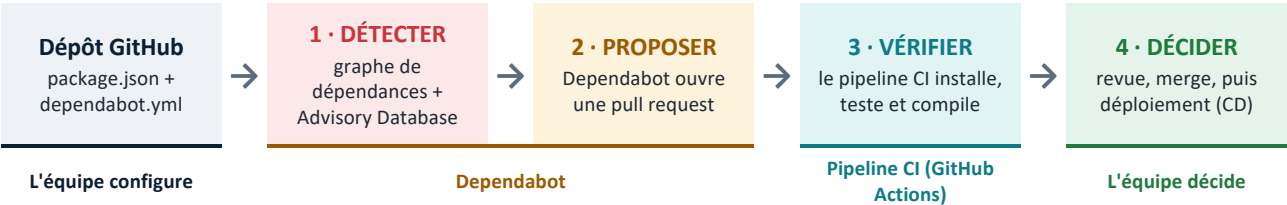
Placé dans .github/dependabot.yml sur la branche par défaut, il active les version updates et permet un réglage fin ; alertes et PR de sécurité fonctionnent sans lui. Voici celui de notre dépôt :

```
version: 2                                # Version du format de ce fichier (toujours 2)
updates:
  # ---- Bloc 1 : dépendances JavaScript (package.json + package-lock.json) ----
  - package-ecosystem: "npm"              # Gestionnaire de paquets à surveiller
    directory: "/"                        # Dossier qui contient Le package.json
    schedule:
      interval: "weekly"                  # Fréquence de vérification : daily, weekly, monthly...
      day: "monday"
      time: "08:00"
      timezone: "Europe/Paris"
    open-pull-requests-limit: 10          # Max de PR de mise à jour de VERSION ouvertes (défaut : 5)
    groups:
      vite:                               # Paquets Liés : mis à jour ENSEMBLE, dans une seule PR
        patterns: ["vite", "@vitejs/*"]  # vite 8 exige @vitejs/plugin-react 6 (et inversement)
      react:                               # react, react-dom et leurs types doivent rester alignés
        patterns: ["react", "react-dom", "@types/react", "@types/react-dom"]
    ignore:
      - dependency-name: "*eslint*"       # Mises à jour que Dependabot ne doit PAS proposer
        update-types: ["version-update:semver-major"]
  # ---- Bloc 2 : actions utilisées par le pipeline CI (.github/workflows/ci.yml) ----
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

La limite de PR est portée à 10 car celle par défaut (5) bloquait la PR d'axios ; les groups réunissent des paquets qui ne fonctionnent qu'ensemble (section 3.1). Autres options utiles : allow, labels, assignees, commit-message, target-branch, cooldown, registries. Pousser une modification du fichier relance une vérification : six secondes plus tard dans notre dépôt, le 23/09/2026.

3. Dependabot dans un pipeline CI/CD

3.1 Détecter, proposer, vérifier, décider



Dependabot n'exécute ni tests ni build et ne déploie rien : son travail s'arrête à l'ouverture de la PR. Comme toute PR, celle-ci déclenche le pipeline d'intégration continue (ici un workflow GitHub Actions sur l'événement pull_request)

qui installe, teste et compile. L'équipe ne fusionne que si le pipeline est vert ; le déploiement continu (CD) prend le relais.

Pourquoi le CI est indispensable : la preuve par notre dépôt. En rejouant localement notre pipeline (`npm ci`, `npm test`, `npm run build`) sur les 7 PR de Dependabot, 3 passent et 4 échouent dès l'installation, sur un conflit de « dépendances pairs » (erreur ERESOLVE). Même la PR de sécurité n° 6 : pour corriger vite, elle le fait passer de la version 5 à la 8, que le plugin `@vitejs/plugin-react` 4 installé ne supporte pas ; fusionnée sans CI, elle aurait cassé l'installation. Les groups de notre configuration font monter ensemble ces paquets liés : ainsi testées, les mises à jour passent le pipeline.

Bonnes pratiques. Protéger la branche principale (« Require status checks to pass before merging ») ; savoir que les workflows déclenchés par Dependabot reçoivent un `GITHUB_TOKEN` en lecture seule, sans accès aux secrets GitHub Actions ; réserver la fusion automatique (`dependabot/fetch-metadata` puis `gh pr merge --auto`) aux mises à jour mineures et correctives, avec de bons tests.

3.2 Limites et points de vigilance

- **Le bruit.** Notre projet de 14 dépendances a généré 12 mises à jour dès le premier passage ; trop de PR finissent par ne plus être lues. Remèdes : `groups`, rythme hebdomadaire, `open-pull-requests-limit`, `ignore`.
- **Des mises à jour cassantes, même de sécurité** (vite 5 → 8 dans la PR n° 6) : Dependabot ne garantit pas la compatibilité.
- **Des alertes pas toujours pertinentes.** La détection compare des numéros de version, pas l'usage réel : plusieurs failles d'axios visent son adaptateur HTTP pour Node.js, alors que notre application l'utilise dans le navigateur. À l'équipe de trier, voire de fermer l'alerte avec une justification (*dismiss*).
- **Une couverture partielle.** Un script chargé depuis un CDN, du code copié-collé ou une faille pas encore publiée (zero-day) échappent à l'analyse. Sans version corrigée, il y a une alerte mais pas de PR ; en cas de conflit, la mise à jour échoue (« Security update failed for esbuild 0.21.5 » dans nos journaux, contourné par la PR n° 7 qui met à jour vite).
- **Nouvelle version ne veut pas dire version sûre.** Le 31 mars 2026, des attaquants ont volé les identifiants npm du mainteneur principal d'axios : les versions 1.14.1 et 0.30.4, publiées avec une dépendance malveillante (`plain-crypto-js`) qui installait un cheval de Troie, sont restées en ligne environ 3 heures. Depuis le 14 juillet 2026, Dependabot attend par défaut 3 jours avant de proposer une nouvelle version ; les correctifs de sécurité restent immédiats.

Conclusion

Dependabot automatise une tâche que personne ne fait bien à la main : surveiller en continu des centaines de dépendances et proposer leur mise à jour. Il fonctionne selon deux modes complémentaires, **sécurité** (réactif, sans configuration) et **versions** (planifié, via `dependabot.yml`), et se règle pour limiter le bruit et le risque (`groups`, `ignore`, `open-pull-requests-limit`, `cooldown`). Mais il **détecte et propose** sans rien tester ni fusionner seul : dans notre dépôt, 4 de ses 7 PR auraient cassé l'installation. Sa valeur dépend donc du **pipeline CI** qui vérifie chaque PR et de **l'équipe** qui décide. Propre à GitHub, il ne remplace ni l'analyse du code de l'application ni les tests : c'est le premier maillon, gratuit et simple à mettre en place, d'une démarche plus large de sécurité de la chaîne d'approvisionnement logicielle.

Sources (consultées le 25 septembre 2026)

- [1] GitHub Docs : « Dependabot security updates », « Dependabot version updates », « About Dependabot alerts », « Dependabot options reference », « Automating Dependabot with GitHub Actions » (docs.github.com/en/code-security).
- [2] GitHub Changelog, 14/07/2026 : « Dependabot version updates introduce default package cooldown ».
- [3] GitHub Advisory Database : avis concernant axios, dont GHSA-jr5f-v2jv-69x6 (CVE-2025-27152) et GHSA-42xw-2xvc-qx8m (CVE-2019-10742).
- [4] axios, « Post Mortem: axios npm supply chain compromise » (github.com/axios/axios/issues/10636), 2026.
- [5] U.S. House Committee on Oversight and Government Reform, « The Equifax Data Breach », décembre 2018.
- [6] Help Net Security, « GitHub introduces Dependabot-powered automated security fixes », 28/05/2019.
- [7] Dépôt de démonstration : PR n° 1 à 7 et journaux « Dependabot Updates » du 23/09/2026 ; tests locaux du pipeline du 25/09/2026.